

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа 6

Выполнил:

Дмитриев Андрей

Группа

К3439

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

1. Задача

1. Подключить и настроить брокер сообщений **RabbitMQ** (допускается Kafka, выбран RabbitMQ).
2. Реализовать межсервисное взаимодействие между сервисами через RabbitMQ.

2. Ход работы

2.1 Добавление RabbitMQ в docker-compose.yml

В docker-compose добавлен контейнер RabbitMQ с Management UI для удобства контроля очередей и сообщений (Листинг 1).

```
# docker-compose.yml
```

```
services:
```

```
# ... existing services ...
```

```
rabbitmq:
```

```
  image: rabbitmq:3-management
```

```
  container_name: school_site_rabbitmq
```

```
  environment:
```

```
    RABBITMQ_DEFAULT_USER: '${RABBITMQ_USER}'
```

```
    RABBITMQ_DEFAULT_PASS: '${RABBITMQ_PASSWORD}'
```

```
  restart: unless-stopped
```

```
  ports:
```

```
    - "5672:5672" # AMQP порт
```

```
    - "15672:15672" # Management UI
```

```
  networks:
```

- shool_site-network

volumes:

- rabbitmq_data:/var/lib/rabbitmq

volumes:

rabbitmq_data:

Листинг 1 — docker-compose.yml (добавление RabbitMQ)

2.2 Настройка подключения к RabbitMQ в основном сервисе

В settings.py добавлена конфигурация RabbitMQ (host/port/user/password/vhost) и вычисляемый URL подключения (Листинг 2).

school_site/settings.py

```
from pydantic import BaseModel
```

```
class RabbitMQ(BaseModel):
```

```
    """
```

```
    Настройки для подключения к RabbitMQ.
```

```
    """
```

```
    host: str
```

```
    port: int = 5672
```

```
    user: str
```

```
    password: str
```

```
    vhost: str = '/'
```

```
    @property
```

```
def url(self) -> str:
    return f'amqp://{self.user}:{self.password}@{self.host}:{self.port}/{self.vhost}'
```

```
class Settings(BaseSettings):
```

```
    # ... existing settings ...
```

```
    rabbitmq: RabbitMQ
```

Листинг 2 — school_site/settings.py

2.3 Создание сервиса публикации сообщений (Publisher)

В основном сервисе создан RabbitMQPublisher на базе aio-pika.

Используется единая **topic-exchange** school.events для публикации событий по routing key (Листинг 3).

```
# school_site/core/rabbitmq.py
```

```
import aio_pika
```

```
import json
```

```
import logging
```

```
from typing import Protocol
```

```
from school_site.settings import settings
```

```
logger = logging.getLogger(__name__)
```

```
class RabbitMQPublisherProtocol(Protocol):
```

```
    async def publish(self, routing_key: str, message: dict) -> None:
```

...

async def close(self) -> None:

...

class RabbitMQPublisher(RabbitMQPublisherProtocol):

"""

Сервис для публикации сообщений в RabbitMQ.

"""

EXCHANGE_NAME = 'school.events' # единая topic-биржа

def __init__(self):

self.connection = None

self.channel = None

self.exchange = None

async def connect(self) -> None:

"""Установка подключения к RabbitMQ."""

self.connection = await aio_pika.connect_robust(

host=settings.rabbitmq.host,

port=settings.rabbitmq.port,

login=settings.rabbitmq.user,

password=settings.rabbitmq.password,

virtualhost=settings.rabbitmq.vhost

```
)

self.channel = await self.connection.channel()

self.exchange = await self.channel.declare_exchange(

    self.EXCHANGE_NAME,

    aio_pika.ExchangeType.TOPIC,

    durable=True

)

logger.info(f"Connected to RabbitMQ, exchange: {self.EXCHANGE_NAME}")
```

```
async def publish(self, routing_key: str, message: dict) -> None:
```

```
    """
```

```
    Публикация сообщения в очередь.
```

```
    """
```

```
    if not self.exchange:
```

```
        await self.connect()
```

```
    body = json.dumps(message, default=str).encode()
```

```
    await self.exchange.publish(
```

```
        aio_pika.Message(
```

```
            body=body,
```

```
            content_type='application/json',
```

```
            delivery_mode=aio_pika.DeliveryMode.PERSISTENT
```

```
        ),
```

```
        routing_key=routing_key
```

```
    )
```

```
logger.info(f"Published message to {routing_key}: {message}")
```

```
async def close(self) -> None:
```

```
    """Закрытие подключения."""
```

```
    if self.connection:
```

```
        await self.connection.close()
```

```
        logger.info("RabbitMQ connection closed")
```

```
rabbitmq_publisher = RabbitMQPublisher()
```

Листинг 3 — school_site/core/rabbitmq.py

2.4 Публикация события при регистрации пользователя

После успешного создания пользователя публикуется событие `user.registered` (Листинг 4).

```
# school_site/apps/users/use_cases/create_user.py
```

```
from typing import Protocol
```

```
from school_site.core.rabbitmq import rabbitmq_publisher
```

```
from ..schemas import UserWithPhotoReadSchema
```

```
from ..services.users import UserServiceProtocol
```

```
class CreateUserUseCaseProtocol(Protocol):
```

```
    async def __call__(self, access_token: str, user_data: str, image) ->
        UserWithPhotoReadSchema:
```

```
    ...
```

```

class CreateUserUseCase(CreateUserUseCaseProtocol):

    def __init__(self, user_service: UserServiceProtocol):

        self.user_service = user_service


    async def __call__(self, access_token: str, user_data: str, image) ->
    UserWithPhotoReadSchema:

        user = await self.user_service.create_user(access_token, user_data, image)


        await rabbitmq_publisher.publish(

            routing_key='user.registered',

            message={

                'id': str(user.id),

                'email': user.email,

                'name': user.name,

                'role': user.role

            }

        )


        return user

```

Листинг 4 — school_site/apps/users/use_cases/create_user.py

2.5 Публикация события при создании студента

При создании студента публикуется событие student.created, которое будет обработано Email-сервисом (Листинг 5).

school_site/apps/students/use_cases/create_student.py


```

from school_site.core.rabbitmq import rabbitmq_publisher

from ..schemas import StudentCreateSchema, StudentReadSchema


class CreateStudentUseCase:

    def __init__(self, student_service):

        self.student_service = student_service

    async def __call__(self, access_token: str, student: StudentCreateSchema) ->
StudentReadSchema:

        created_student = await self.student_service.create(access_token, student)

        await rabbitmq_publisher.publish(

            routing_key='student.created',

            message={

                'student_id': str(created_student.id),

                'user_id': str(created_student.user_id),

                'event_type': 'student.created'

            }

        )

        return created_student

```

Листинг 5 — Публикация события при создании студента

2.6 Реализация консьюмера (Consumer) в Email-service

В Email-сервисе создан RabbitMQConsumer, который:

- подключается к exchange school.events (тип TOPIC),
- создаёт очередь email.notifications,
- привязывает routing keys к очереди,
- запускает обработку входящих сообщений (Листинг 6).

```
# notification_email_service/core/rabbitmq_consumer.py
```

```
import aio_pika
```

```
import json
```

```
import logging
```

```
from typing import Callable, Dict
```

```
from notification_email_service.settings import settings
```

```
logger = logging.getLogger(__name__)
```

```
class RabbitMQConsumer:
```

```
    """
```

```
    Консьюмер для получения сообщений из RabbitMQ.
```

```
    """
```

```
    EXCHANGE_NAME = 'school.events'
```

```
    QUEUE_NAME = 'email.notifications'
```

```
    def __init__(self):
```

```
        self.connection = None
```

```
self.channel = None
```

```
self.handlers: Dict[str, Callable] = {}
```

```
def register_handler(self, routing_key: str, handler: Callable):
```

```
    """Регистрация обработчика для routing key."""
```

```
    self.handlers[routing_key] = handler
```

```
    logger.info(f"Registered handler for {routing_key}")
```

```
async def connect(self) -> None:
```

```
    """Установка подключения к RabbitMQ."""
```

```
    self.connection = await aio_pika.connect_robust(
```

```
        host=settings.rabbitmq.host,
```

```
        port=settings.rabbitmq.port,
```

```
        login=settings.rabbitmq.user,
```

```
        password=settings.rabbitmq.password,
```

```
        virtualhost=settings.rabbitmq.vhost
```

```
    )
```

```
    self.channel = await self.connection.channel()
```

```
    await self.channel.set_qos(prefetch_count=10)
```

```
    exchange = await self.channel.declare_exchange(
```

```
        self.EXCHANGE_NAME,
```

```
        aio_pika.ExchangeType.TOPIC,
```

```
        durable=True
```

```
    )
```

```
queue = await self.channel.declare_queue(  
    self.QUEUE_NAME,  
    durable=True  
)
```

```
for routing_key in self.handlers.keys():  
    await queue.bind(exchange, routing_key)  
    logger.info(f"Queue bound to {routing_key}")
```

```
await queue.consume(self._process_message)  
logger.info(f"Started consuming from {self.QUEUE_NAME}")
```

```
async def _process_message(self, message: aio_pika.IncomingMessage):
```

```
    """Обработка входящего сообщения."""
```

```
    async with message.process():
```

```
        routing_key = message.routing_key
```

```
        payload = json.loads(message.body.decode())
```

```
        logger.info(f"Received message [{routing_key}]: {payload}")
```

```
        handler = self.handlers.get(routing_key)
```

```
        if handler:
```

```
            await handler(payload)
```

```
        else:
```

```
            logger.warning(f"No handler for routing_key: {routing_key}")
```

```
rabbitmq_consumer = RabbitMQConsumer()
```

Листинг 6 — notification_email_service/core/rabbitmq_consumer.py

2.7 Обработчики событий (Email-service)

Реализован обработчик события user.registered, который формирует и отправляет приветственное письмо (Листинг 7).

Также предусмотрен обработчик student.created (пока как заготовка под логику уведомлений).

```
# notification_email_service/apps/emails/handlers/user_registered.py
```

```
import logging
```

```
from notification_email_service.apps.emails.schemas import EmailCreate
```

```
from notification_email_service.apps.emails.services.email_sender import  
EmailSenderService
```

```
logger = logging.getLogger(__name__)
```

```
async def handle_user_registered(payload: dict) -> None:
```

```
    """
```

```
    Обработчик события регистрации пользователя.
```

```
    Отправляет приветственное письмо.
```

```
    """
```

```
    logger.info(f"Handling user.registered event for user: {payload.get('id')}")
```

```
    email = EmailCreate(
```

```
to_email=payload['email'],  
subject='Добро пожаловать в IT School!',  
body=f"""  
Здравствуйте, {payload.get('name', 'Пользователь')}!
```

Ваша регистрация в системе IT School успешно завершена.

Ваша роль: {payload.get('role')}

С уважением,

Команда IT School

"""

)

```
sender = EmailSenderService()  
result = await sender.send_email(email)
```

if result is True:

```
    logger.info(f"Welcome email sent to {payload['email']}")
```

else:

```
    logger.error(f"Failed to send welcome email: {result}")
```

```
async def handle_student_created(payload: dict) -> None:
```

"""

Обработчик события создания студента.

```

"""

logger.info(f"Handling student.created event: {payload}")

# Логика отправки уведомления о создании студента

```

Листинг 7 — notification_email_service/apps/emails/handlers/user_registered.py

2.8 Запуск консьюмера при старте Email-сервиса

Консьюмер стартует в lifecycle FastAPI:

- регистрируются обработчики,
- создаётся фоновая задача connect() (Листинг 8).

```
# notification_email_service/main.py
```

```

import asyncio

from contextlib import asynccontextmanager

from fastapi import FastAPI

from notification_email_service.core.rabbitmq_consumer import rabbitmq_consumer
from notification_email_service.apps.emails.handlers.user_registered import (
    handle_user_registered,
    handle_student_created
)

```

```
@asynccontextmanager
```

```
async def lifespan(app: FastAPI):
```

```

    rabbitmq_consumer.register_handler('user.registered', handle_user_registered)
    rabbitmq_consumer.register_handler('student.created', handle_student_created)

```

```
asyncio.create_task(rabbitmq_consumer.connect())
```

```
yield
```

```
if rabbitmq_consumer.connection:
```

```
    await rabbitmq_consumer.connection.close()
```

```
app = FastAPI(lifespan=lifespan)
```

Листинг 8 — notification_email_service/main.py

3. Архитектура межсервисного взаимодействия (схема)

СХЕМА: ОБМЕН СООБЩЕНИЯМИ ЧЕРЕЗ RABBITMQ

- main-app (FastAPI) публикует события → exchange school.events (topic)
- RabbitMQ маршрутизирует сообщения по routing key
- email-app (FastAPI) потребляет сообщения из очереди email.notifications и выполняет действия (например, отправка email)

УПРОЩЁННАЯ ТЕКСТОВАЯ СХЕМА:

main-app (:8080)

→ publish event (routing_key)

→ RabbitMQ (:5672, exchange school.events)

→ queue email.notifications

→ email-app (:9090) consume

→ запись логов в email_db (:5436)

4. Routing Keys (ключи маршрутизации) — схема вместо таблицы

СХЕМА: ROUTING KEYS

- user.registered — producer: main-app → consumer: email-app (регистрация пользователя)
- user.password.reset — producer: main-app → consumer: email-app (сброс пароля)
- student.created — producer: main-app → consumer: email-app (создание студента)
- student.points.updated — producer: main-app → consumer: email-app (обновление баллов)
- lesson.scheduled — producer: main-app → consumer: email-app (назначение урока)
- event.created — producer: main-app → consumer: email-app (создание события)

5. Переменные окружения (.env)

RabbitMQ

SCHOOL_SITE_APP_RABBITMQ__HOST=rabbitmq

SCHOOL_SITE_APP_RABBITMQ__PORT=5672

SCHOOL_SITE_APP_RABBITMQ__USER=guest

SCHOOL_SITE_APP_RABBITMQ__PASSWORD=guest

SCHOOL_SITE_APP_RABBITMQ__VHOST=/

Для docker-compose

RABBITMQ_USER=guest

RABBITMQ_PASSWORD=guest

Листинг 9 — .env

6. Зависимости проекта

pyproject.toml

```
[project]
dependencies = [
    # ... existing dependencies ...
    "aio-pika>=9.0.0",
]
```

Листинг 10 — pyproject.toml

7. Вывод

В рамках лабораторной работы в микросервисную архитектуру был внедрён брокер сообщений **RabbitMQ** для асинхронного межсервисного взаимодействия.

Реализовано следующее:

1. **Main-service** публикует события (например `user.registered`, `student.created`) в topic-exchange `school.events`.
2. **Email-service** запускает consumer, подписывается на нужные routing keys и выполняет действия (например, отправка приветственного письма), а также может логировать операции в собственной БД.

Преимущества использования RabbitMQ в данной системе:

- асинхронность (main-app не ждёт отправки email);
- надёжность (durable exchange/queue и persistent сообщения);
- масштабируемость (можно запускать несколько consumer);
- слабая связность сервисов (нет прямых синхронных HTTP-вызовов между сервисами).